

Cliente-Servidor TCP

Objetivo

Compreender na prática o modelo cliente-servidor utilizando o protocolo TCP/IP. O aluno irá iniciar dois containers em uma rede virtual Docker, observar a comunicação TCP, enviar mensagens entre os containers e simular situações de falha de rede.

Pré-requisitos

- Docker e Docker Compose instalados
- Conhecimento básico de terminal Linux
- Noções de endereçamento IP e portas

Como iniciar o laboratório

Abra um terminal nesta pasta e execute:

```
docker compose up -d
```

Em seguida, entre no container do cliente (Terminal 1):

```
docker compose exec cliente bash --login
```

Este será o terminal principal do aluno durante o laboratório.

Exercícios

Exercício 1 — Conectar o cliente ao servidor

No Terminal 1 (dentro do container), execute:

```
python3 client.py
```

Experimente enviar as seguintes mensagens e observe as respostas:

- oi
- hora
- qualquer texto livre (será ecoado de volta)

- sair para encerrar a conexão

Exercício 2 — Observar os logs do servidor

Abra um Terminal 2 nesta pasta (fora do container) e execute:

```
docker compose logs -f servidor
```

Volte ao Terminal 1, execute `python3 client.py` novamente e envie mensagens. Observe os logs aparecendo em tempo real no Terminal 2.

Exercício 3 — Verificar a rede entre os containers

No Terminal 1 (dentro do container), verifique as interfaces de rede:

```
ip addr
```

Teste a conectividade com o servidor via ping:

```
ping servidor
```

Use Ctrl+C para interromper o ping.

Exercício 4 — Capturar tráfego com tcpdump

No Terminal 1 (dentro do container), inicie a captura de tráfego TCP na porta 9000:

```
tcpdump -i eth0 port 9000 -v
```

Abra um Terminal 3 nesta pasta e entre no container novamente:

```
docker compose exec cliente bash --login
```

Execute o cliente Python nesse novo terminal:

```
python3 client.py
```

Envie mensagens e observe o handshake TCP (SYN, SYN-ACK, ACK) sendo capturado no Terminal 1.

Exercício 5 — Simular queda do servidor

No Terminal 1, execute o cliente Python:

```
python3 client.py
```

Com o cliente em execução, abra um Terminal 2 nesta pasta e pare o servidor:

```
docker compose stop servidor
```

Observe no Terminal 1 o que acontece com a conexão TCP quando o servidor cai.

Depois reinicie o servidor no Terminal 2:

```
docker compose start servidor
```

E tente conectar novamente no Terminal 1:

```
python3 client.py
```

Exercício 6 — Conectar usando netcat

No Terminal 1 (dentro do container), conecte diretamente com netcat, sem usar o script Python:

```
nc servidor 9000
```

Digite mensagens manualmente e observe as respostas do servidor. Use Ctrl+C para encerrar.

Como encerrar o laboratório

No Terminal 1, saia do container:

```
exit
```

No Terminal 2, derrube os containers:

```
docker compose down
```